

Malak Fathy Abdelaal Ali 20237014

Farah Amr Salah 20237021

Mennatullah Mohamed 20236143

Alaa Mohamed Taher 20237002

Data Preparation & Preprocessing

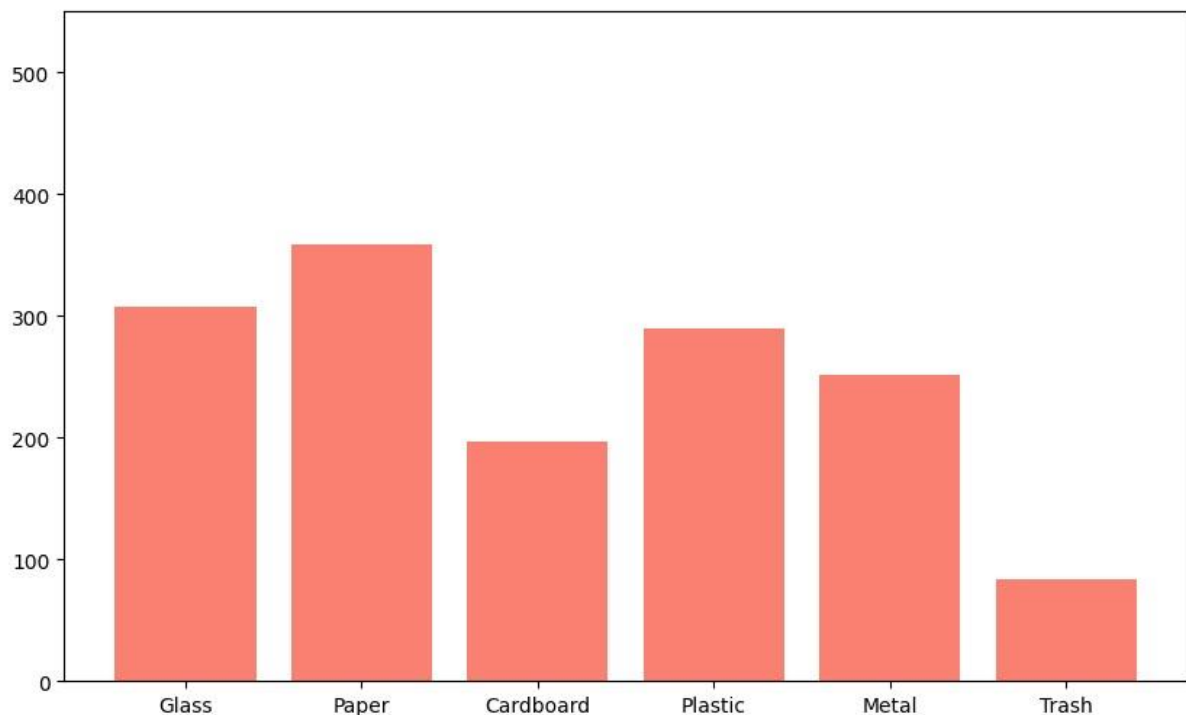
1. Dataset Overview

The primary goal of this phase was to transform the raw **Garbage Classification** dataset into a standardized, balanced, and high-quality format suitable for training SVM and k-NN classifiers. The dataset consists of six material categories: **Glass, Paper, Cardboard, Plastic, Metal, and Trash**.

Upon initial inspection, the dataset was found to be significantly imbalanced, with the "Trash" class containing only under 100 samples compared to "Paper" with over 350. To ensure the final model does not develop a bias toward the majority classes, a target of **500 images per class** was set for the training set.

Class ID	Class Name	Count	Needed for 500
0	Glass	308	192
1	Paper	359	141
2	Cardboard	197	303
3	Plastic	290	210
4	Metal	252	248
5	Trash	84	416

Total images in Training: 1490			



2.Preprocessing Methodology

A. Data Cleaning and Integrity

- Corruption Check: Every image was passed through PIL.verify() to detect and remove truncated or broken files that could crash the training script.
- Deduplication: To prevent overfitting, an MD5 Hashing algorithm was used to scan for bit-identical files. Duplicate images were removed to ensure the model learns generalized features rather than memorizing specific pixel patterns.

B. Image Standardization

- Color Space: All images were converted to 3-channel RGB. This prevents dimensional errors if the raw data contained Grayscale or RGBA files.

- Resizing: Images were downsampled to 128x128 pixels using the LANCZOS interpolation filter
- Normalization: Pixel intensities were scaled from [0, 255] to the [0, 1.0] range. This "Feature Scaling" is critical for distance-based models like k-NN, ensuring that no single pixel value dominates the calculation.

3. Augmentation Methods

- Rotation: Random rotations (up to 30°) were applied.
- Horizontal Flipping: A 50% probability "coin-flip" mirror effect was used. This ensures the model is invariant to the left-to-right orientation of objects.
- Scaling (Zoom In/Out): Using center-cropping and black-background padding, we simulated the object being closer to or further from the camera lens.
- Brightness Jitter: Intensities were varied by +20 and -20.

Feature Extraction and Descriptor Comparison

1. Overview

We used three feature descriptors: HOG, LBP, and Color Histogram, each capturing different image characteristics for material classification.

2. HOG (Histogram of Oriented Gradients)

HOG captures shape and edge information by analyzing gradient orientations.

Parameters: orientations = 9, pixels per cell = (16×16), cells per block = (2×2)

The (16×16) cell size outperformed (8×8) by reducing noise and improving generalization.

HOG is effective for materials with clear structural patterns (e.g., metal edges, cardboard geometry).

3. LBP (Local Binary Patterns)

LBP captures local texture patterns using neighborhood intensity comparisons.

Parameters: radius = 3, points = 24 (uniform)

While useful for texture, it lacks global structure, making it less suitable for distinguishing materials with similar textures but different shapes.

4. Color Histogram

Color histograms represent the distribution of colors in an image.

They help differentiate materials based on appearance (e.g., cardboard vs plastic), but ignore spatial and structural information.

5. Combined Features (HOG + Color Histogram)

Combining HOG with color histograms yielded the best performance:

SVM accuracy: 0.712

This combination leverages both structure (HOG) and color (histogram), resulting in a richer and more discriminative feature representation.

6. Detailed Comparison

HOG vs LBP:

HOG significantly outperformed LBP:

HOG (KNN): ~0.563

LBP (SVM): ~0.53

LBP (KNN): ~0.485

This is because material classification depends more on global shape and edges than fine texture. For example, metal and plastic may share similar surface textures, but differ in structure and boundaries, which HOG captures and LBP does not.

LBP Limitations:

LBP relies only on local pixel patterns, so it struggles when different materials have similar textures or when lighting conditions vary. This leads to weaker feature discrimination and lower accuracy.

HOG vs HOG + Color Histogram:

HOG alone captures structure but ignores color, which is an important cue in this dataset. Adding color histograms improved performance to:

0.712 (SVM with HOG + Color Histogram)

This improvement occurs because some materials are better distinguished by color differences (e.g., brown cardboard vs transparent glass), which HOG cannot represent.

Why HOG + Color is Best for This Project:

Material classification requires both:

Structural information (shape, edges → captured by HOG)

Appearance information (color → captured by histogram)

The combination reduces ambiguity between visually similar classes and provides a more complete representation of each material.

Classifier Implementation & Results

1 Support Vector Machine (SVM)

We used an **RBF kernel** because the features are not linearly separable, and RBF handles curved boundaries well. We set **C = 10** and **gamma = 'scale'**.

SVM + HOG + LBP (Combined Features): PCA with 500 components was applied to reduce the feature size.

- **Accuracy: 71.2%**

- Paper and Cardboard were the easiest to classify (F1 = 0.80,0.81)
 - Trash was the hardest (F1 = 0.48) because it has very few and diverse samples
-

2 k-Nearest Neighbors (k-NN)

We used **k = 5** and **distance-based weighting**, so closer neighbors have more influence on the prediction than farther ones. PCA with 500 components was also applied before training.

KNN + HOG + LBP (Combined Features): PCA with 500 components was applied to reduce the feature size.

- **Accuracy: 58.4%**
 - Paper was the best class (F1 = 0.71), Trash was the worst (F1 = 0.43)
-

3 Unknown Class Rejection

If the model's confidence is below **0.6**, the image is labeled as **Unknown (class 6)** instead of forcing a wrong prediction.

Model	Classified	Rejected as Unknown
SVM combined	278 / 375	97
KNN combined	231 / 375	144

We first tried HOG alone and LBP alone with both classifiers. HOG alone gave better results than LBP alone, but both were weaker than the combined version. Combining HOG + Color + LBP together gave the best accuracy for both SVM and k-NN, so we used it as our final feature vector.

SVM vs k-NN: SVM was clearly better than k-NN. SVM finds a strong decision boundary that generalizes well, while k-NN struggles when the feature vectors are large and complex.

Best Model: SVM + HOG +LBP (Combined) with 71.2% accuracy used for the real-time camera system.

Real-Time Camera Loop

The camera loop integrates all components into a continuously running application that captures frames, classifies the item inside the ROI, and displays the annotated result in real time.

1 Region of Interest

A fixed 250x250 pixel bounding box is positioned at coordinates (200, 100) on the frame. Only the pixels within this region are passed to the feature extractor. The user places the item inside this box, which reduces background noise and ensures consistent input framing.

2 Per-Frame Processing Sequence

Step	Operation
1	cap.read() captures the current webcam frame
2	ROI is cropped using numpy array slicing
3	extract_features(roi) produces the 500-dimensional PCA feature vector
4	predict_with_rejection() returns class ID and confidence score
5	Label string is constructed from class name and confidence
6	Rectangle and text labels are drawn onto the full frame
7	cv2.imshow() renders the annotated frame in the display window

3 Visual Feedback

The classification label is rendered just above the ROI box. Green text indicates a confident prediction; red text indicates the item was rejected as Unknown. A static instruction message is shown in the top-left corner on every frame.

4 Exit Handling

Two exit methods are supported. Pressing Q triggers a break via `cv2.waitKey()`. Clicking the window X button is detected by monitoring `cv2.WND_PROP_VISIBLE` — when the window is closed the property drops below 1 and the loop breaks. In both cases `cap.release()` and `cv2.destroyAllWindows()` execute to cleanly release the camera and close the display.

Training vs. Deployment Pipeline Consistency

The table below maps every training preprocessing step to its exact deployment equivalent, demonstrating full pipeline consistency.

Training Step	Deployment Equivalent
<code>img.resize((96,96), LANCZOS)</code>	<code>preprocess_image()</code> PIL resize to (96, 96) LANCZOS
<code>np.array(img) / 255.0</code>	<code>preprocess_image()</code> <code>astype float32 / 255.0</code>
<code>rgb2gray(img)</code>	<code>preprocess_image()</code> <code>cv2.COLOR_RGB2GRAY / 255.0</code>
<code>hog(orientations=9, pixels_per_cell=(16,16), block_norm=L2-Hys)</code>	<code>extract_features()</code> identical <code>hog()</code> call
<code>local_binary_pattern(img, 24, 3, uniform)</code>	<code>extract_features()</code> identical <code>lbp</code> call
<code>np.histogram(bins=32, range=(0,1))</code>	<code>extract_features()</code> identical histogram call
<code>sc_hog.fit_transform(X_train_hog)</code>	<code>extract_features()</code> <code>sc_hog.transform([hog_feat])</code>
<code>sc_color.fit_transform(color_train)</code>	<code>extract_features()</code> <code>sc_color.transform([color_feat])</code>

<code>sc_lbp.fit_transform(X_train_lbp)</code>	<code>extract_features() sc_lbp.transform([lbp_feat])</code>
<code>np.hstack([hog, color, lbp])</code>	<code>extract_features() np.hstack([hog_s, color_s, lbp_s])</code>
<code>pca.fit_transform(X_train_combined)</code>	<code>extract_features() pca.transform(combined)</code>

Any deviation in this mapping — including loading a scaler or PCA from a different training session, or modifying a parameter such as `pixels_per_cell` — will silently produce incorrect predictions without raising an error. All five saved artifacts must always originate from the same training run.

Git Rep: <https://github.com/malakfathyabdelal/Material-Stream-Identification-System>